



MOBILNE APLIKACIJE

Vežbe 4

GUI II

2025/2026

Sadržaj

1. ViewBinding	3
2. Adapteri	4
2.1 Pravljenje adaptera	4
3. Toolbar	8
3.1 Pravljenje toolbar-a	8
4. Navigation Drawer	13
5. Toasts / Snackbars	18
6. Spinner	19
7. Dijalozi	21
8. View Model	22
8.1 Perzistencija (Persistence)	22
8.2 SavedStateHandle	22
8.3 Opseg (Scope)	23
8.4 Životni ciklus ViewModel-a	23
8.5 Pristup poslovnoj logici	24
8.6 Implementacija	25
9. Domaći	28

1. ViewBinding

ViewBinding je funkcionalnost u Androidu koja olakšava interakciju s prikazima. Generiše binding klasu za svaki XML fajl sa rasporedom elemenata u vašoj aplikaciji. Ova binding klasa omogućava direktni pristup elementima prikaza (views) u fajlu rasporeda, bez potrebe za korišćenjem *findViewById* metode.

Da bismo koristili ViewBinding u Android aplikaciji, neophodno je da omogućimo ViewBinding u *build.gradle* fajlu unutar *android{}*.

```
34      buildFeatures { ApplicationBuildFeatures it ->
35          viewBinding true
36      }
```

Slika 1. build.gradle - omogućavanje ViewBinding-a

Za svaki layout XML fajl sa rasporedom elemenata, generiše se odgovarajuća *ViewBinding* klasa. Na primer, ako je vaš fajl rasporeda nazvan *activity_home.xml*, generisana klasa će biti *ActivityHomeBinding*.

U vašoj Aktivnosti ili Fragmentu, napravite inflaciju rasporeda elemenata pomoću generisane *ViewBinding* klase:

```
28      @Override
29      protected void onCreate(Bundle savedInstanceState) {
30          super.onCreate(savedInstanceState);
31
32          binding = ActivityHomeBinding.inflate(getLayoutInflater());
33          setContentView(binding.getRoot());
34
35          setupUI();
36          setupNavigation();
37      }
```

Slika 2. Inflacija rasporeda elemenata pomoću generisane ViewBinding klase u aktivnosti

```
42      binding.activityHomeBase.floatingActionButton.setOnClickListener( View v ->
43          Intent intent = new Intent(packageContext.this, CartActivity.class);
44          intent.putExtra(name: "title", value: "Cart");
45          startActivity(intent);
46      });
```

Slika 3. Pristupanje elementima korišćenjem binding objekta

2. Adapteri

Adapteri povezuju poglede i izvore podataka (podaci iz baze, sa interneta..).

Moguće je koristiti neke od predefinisanih adaptera (*BaseAdapter*, *ArrayAdapter*, *CursorAdapter*) ili napraviti *Custom Adapters*, adaptere koji povezuju proizvoljan pogled i izvor podataka.

ArrayAdapter povezuju *TextView* pogled (ili pogled koji sadrži *TextView* pogled) i niz ili kolekciju. Automatski se poziva *toString()* metoda svakog objekta u nizu ili kolekciji i njena povratna vrednost se prikazuje u pogledu.

2.1 Pravljenje adaptera

Kreiramo novu klasu, *ProductListAdapter*, koja nasleđuje neki od postojećih adaptera. U našem primeru nasleđuje *RecyclerView* (slika 4), koji koristi *ProductViewHolder* za opis izgleda pojedinačnog elementa u listi. Dobijamo metode koje moramo da redefinišemo, da bi naš adapter ispravno radio.

```
18 public class ProductsListAdapter extends RecyclerView.Adapter<ProductsListAdapter.ProductViewHolder> {
19
20     5 usages
21     private final List<Product> products = new ArrayList<>();
22
23     1 usage
24     public ProductsListAdapter(List<Product> initialProducts) {
25         if (initialProducts != null) {
26             products.addAll(initialProducts);
27         }
28     }
29 }
```

Slika 4. Kreiranje adaptera

Da bismo mogli prikazati podatke jednog produkta u listi, potrebno je da povežemo *xml* prikaza produkta sa podacima određenog produkta iz liste proizvoda. Funkcija *onCreateViewHolder()* mapira XML layout (*product_card.xml*) u pravi *view* objekat. On se prosleđuje *ProductViewHolder*-u koji ga čuva i omogućava pristup pojedinačnim elementima unutar layouta (Slika 5).

```
28 @NonNull
29 @Override
30 public ProductViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
31     View itemView = LayoutInflater.from(parent.getContext())
32         .inflate(R.layout.product_card, parent, attachToRoot: false);
33     return new ProductViewHolder(itemView);
34 }
```

Slika 5. Povezivanje xml elementa sa podacima jednog produkta

Na slici 6 vidimo kako izgleda mapiranje pomoću *ProductViewHolder*-a.

```
57     public static class ProductViewHolder extends RecyclerView.ViewHolder {
58         public final TextView name;
59         public final TextView description;
60         public final ImageView image;
61
62         public ProductViewHolder(@NonNull View itemView) {
63             super(itemView);
64             name = itemView.findViewById(R.id.product_title);
65             description = itemView.findViewById(R.id.product_description);
66             image = itemView.findViewById(R.id.product_image);
67         }
68     }
```

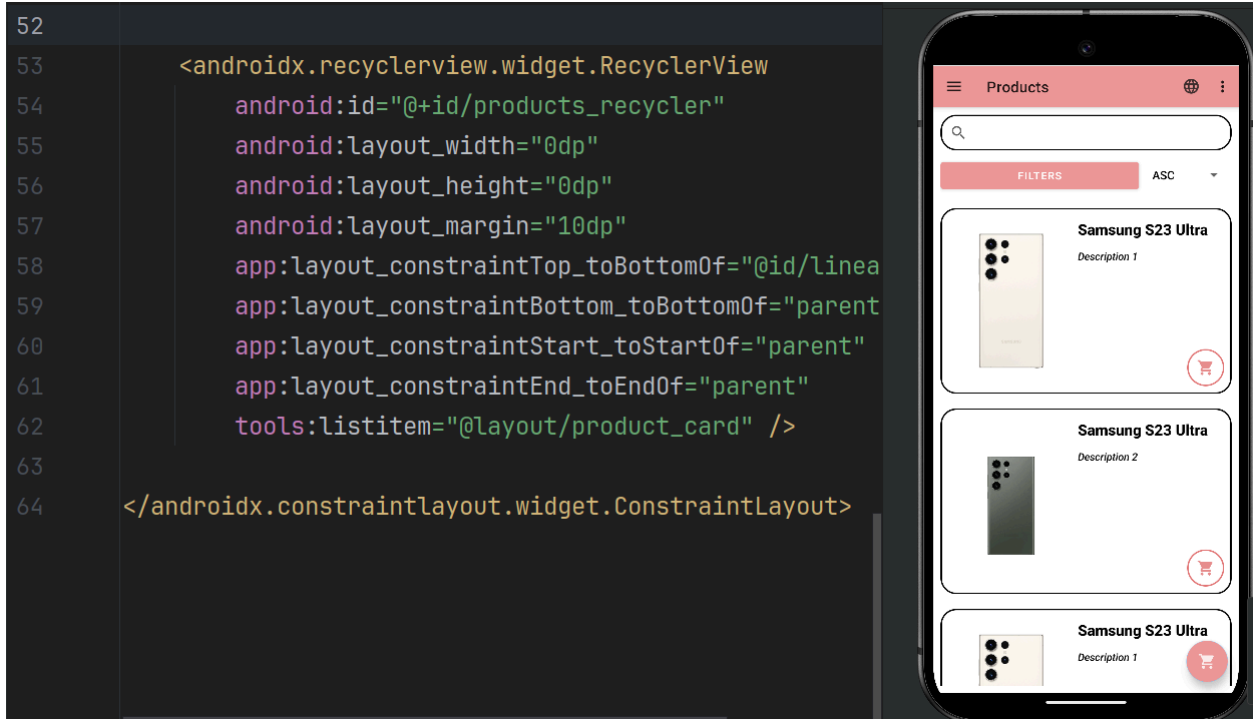
Slika 6. *ProductViewHolder*

Metoda *onBindViewHolder()* povezuje konkretne podatke jednog proizvoda na određenoj poziciji unutar liste i prikazuje ih unutar odgovarajućih UI elemenata definisanih u holderu (slika 7).

```
36     @Override
37     public void onBindViewHolder(@NonNull ProductViewHolder holder, int position) {
38         Product product = products.get(position);
39         holder.name.setText(product.getTitle());
40         holder.description.setText(product.getDescription());
41         holder.image.setImageResource(product.getImage());
42     }
```

Slika 7. Popunjavanje prikaza jednog produkta

Definisanje *RecyclerView*-a prikazano je na slici 8. *tools:listitem* predstavlja izgled jedne kartice proizvoda. Na emulatoru je prikazan izgled početne stranice gde se vidi ceo view sa karticama pojedinačnih proizvoda. Svaki proizvod ima svoju sliku, naziv i opis.



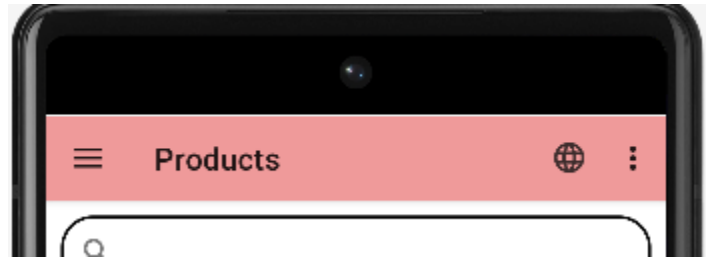
Slika 8. *RecyclerView* komponenta

3. Toolbar

Toolbar je element GUI-a koji se uglavnom nalazi na vrhu ekrana i obezbeđuje navigaciju, izvršavanje akcija, promenu pogleda, *branding* aplikacije.

3.1 Pravljenje toolbar-a

Na slici 11 se nalazi primer *toolbar*-a.



Slika 11. Primer *toolbar*-a

Toolbar se nalazi unutar *AppBarLayout*-a, tj. unutar elementa:

`<com.google.android.material.appbar.AppBarLayout>`.

Otvoriti *activity_home_base.xml* layout. *AppBarLayout* je vertikalni *LinearLayout* koji obezbeđuje mnoga svojstva *Material design*-a. *AppBarLayout layout* se koristi kao dete *CoordinatorLayout*-a.

Material design je skup principa za vizuelni dizajn, dizajn pokreta i dizajn interakcija. Sve aplikacije koje su dizajnirane po ovim principima pružaju korisnicima konzistentno iskustvo i obezbeđuju da korišćenje aplikacija bude intuitivno.

Toolbar možemo ručno da definišemo ili u *Design* režimu da ga prevučemo iz palete (Containers > Toolbar) element `<androidx.appcompat.widget.Toolbar>` (slika 12).

```

11      <com.google.android.material.appbar.AppBarLayout
12          android:id="@+id/app_bar"
13          android:layout_width="match_parent"
14          android:layout_height="wrap_content"
15          app:layout_constraintEnd_toEndOf="parent"
16          app:layout_constraintStart_toStartOf="parent"
17          app:layout_constraintTop_toTopOf="parent"
18          app:layout_constraintBottom_toBottomOf="parent"
19          app:layout_constraintVertical_bias="0.001"
20          android:theme="@style/AppTheme.AppBarOverlay">
21
22      <androidx.appcompat.widget.Toolbar
23          android:id="@+id/toolbar"
24          android:layout_width="match_parent"
25          android:layout_height="?attr/actionBarSize"
26          android:background="?attr/colorPrimary"
27          app:popupTheme="@style/AppTheme.PopupOverlay"
28          app:layout_scrollFlags="scroll|enterAlways"
29          app:menu="@menu/toolbar_menu">
30
31      </androidx.appcompat.widget.Toolbar>
32
33  </com.google.android.material.appbar.AppBarLayout>

```

Slika 12. Kreiranje *Toolbar*-a

Sledeći korak je da kreiramo klasu `HomeActivity.java`, koja nasleđuje `AppCompatActivity`. U metodi `onCreate()` dobavljamo *toolbar* (linija 76) i pozivamo metodu `setSupportActionBar()` kojoj prosleđujemo dobavljeni *toolbar* (slika 13). `ActionBar` je standardni element korisničkog interfejsa u Android aplikacijama koji prikazuje naslov aplikacije, ikone i druge korisničke akcije na vrhu ekrana.

```

40      getSupportActionBar(binding.activityHomeBase.toolbar);

```

Slika 13. Postavljenje *Toolbar*-a

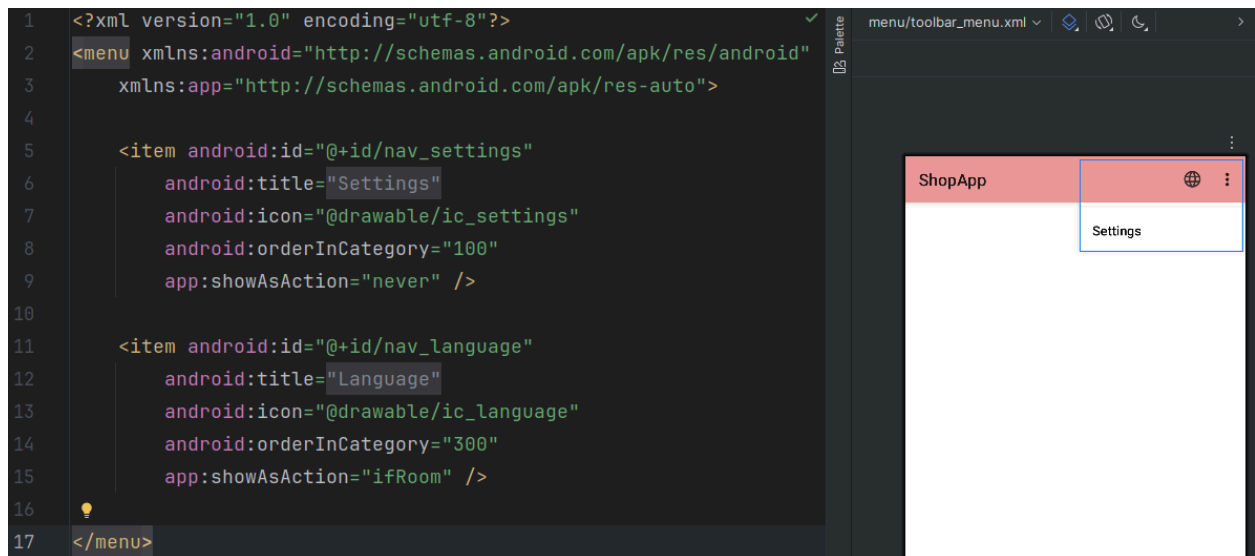
3.2 Postavljanje ikonica na toolbar i povezivanje sa akcijama

Da bismo mogli da izvršavamo akcije klikom na određene dugmiće iz *toolbar*-a, prvo kreiramo meni kao resurs. U ovom primeru kreirali smo meni sa 2 stavke: *settings* i *language* (slika 14) i to će biti ikonice koje će se prikazati na *toolbar*-u. Za svaku stavku smo definisali atribut *app:showAsAction*. Ovaj atribut govori kad i kako će stavka menija da bude prikazana.

Vrednosti koje ovaj atribut može da ima su:

- *ifRoom*
- *withText*
- *never*
- *always*
- *collapseActionView*

Ako atribut postavimo na *ifRoom* to znači da će ta stavka menija biti prikazana samo ako postoji dovoljno prostora u meniju. U suprotnom ova stavka će biti smeštena u padajući meni, kao što smo za stavku *settings* eksplicitno uradili sa postavljanjem atributa na vrednost *never*.



Slika 14. Menu stavke

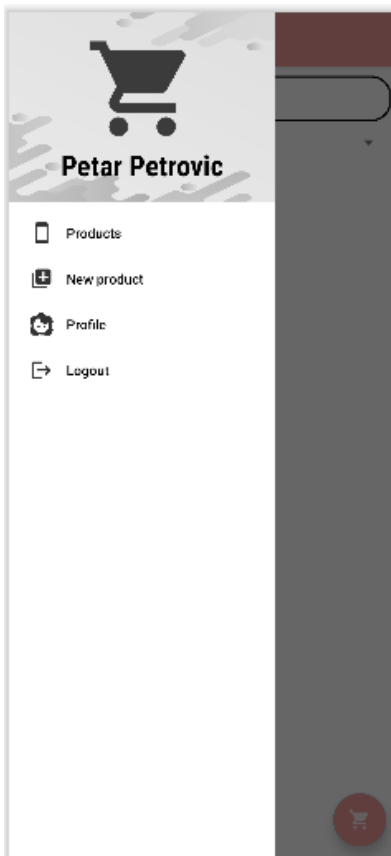
Da bismo kreirali meni postavili na *toolbar*, redefinišemo metodu *onCreateOptionsMenu* u *HomeActivity.java*. Za povezivanje dugmića iz *toolbar*-a sa akcijama, koristimo metodu *onOptionsItemSelected* (slika 16) u *HomeActivity.java*. Ova metoda vraća *MenuItem* na koji je korisnik kliknuo. Taj *MenuItem* u sebi sadrži i identifikator, uz pomoć kojeg znamo tačno na koje dugme iz *toolbar*-a smo kliknuli (slika 15).

```
108         @Override
109         public boolean onCreateOptionsMenu(Menu menu) {
110             getMenuInflater().inflate(R.menu.toolbar_menu, menu);
111             return true;
112         }
113
114         @Override
115         public boolean onOptionsItemSelected(@NonNull MenuItem item) {
116             return NavigationUI.onNavDestinationSelected(item, navController)
117                 || super.onOptionsItemSelected(item);
118         }
119     }
120 }
```

Slika 15. Rad sa menijem

4. Navigation Drawer

Na slici 17 se nalazi primer *Navigation Drawer*-a koji ćemo kreirati.



Slika 17. Primer *Navigation Drawer*-a

Prvo deklariramo *DrawerLayout* raspored (slika 18) i u njega smestamo ostale poglede (*activity_home.xml*). Dodajemo jedan pogled koji sadrži glavni sadržaj aktivnosti *activity_home_base* i drugi pogled koji sadrži komponentu za navigaciju *NavigationView* (*nav_view*).

NavigationView je deo Android korisničkog interfejsa koji se nalazi u *Android Design Support Library*. Obično se koristi unutar bočnog menija (navigation drawer) kako bi omogućio korisnicima navigaciju do različitih delova aplikacije.

Osnovna svrha *NavigationView*-a je da prikaže skup stavki unutar bočnog menija. Svaka stavka može biti povezana s određenom destinacijom unutar aplikacije, na primer, sa fragmentom ili drugom aktivnošću.

android:layout_gravity="start": Ovo postavlja gravitaciju *NavigationView*-a na start, što obično znači da će se bočni meni prikazivati sa leve strane ekrana.

android:fitsSystemWindows="true": Ovo se koristi da bi se osiguralo da se sadržaj prilagodi oko sistema prozora kako bi se izbegli prekidi ili isprekidani prikazi.

app:headerLayout="@layout/nav_header": Ovde se postavlja referenca na layout koji će se koristiti kao zaglavlje (header) unutar NavigationView-a. U ovom slučaju, @layout/nav_header se koristi za postavljanje zaglavlja koje se obično koristi za prikazivanje informacija o korisniku ili dodatnih opcija.

app:menu="@menu/nav_menu": Ovo postavlja referencu na menu resurs (nav_menu) koji sadrži stavke menija koje će biti prikazane unutar NavigationView-a. nav_menu obično definiše različite opcije menija koje korisnici mogu odabrati prilikom navigacije kroz aplikaciju.

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.drawerlayout.widget.DrawerLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      xmlns:tools="http://schemas.android.com/tools"
6      android:layout_width="match_parent"
7      android:layout_height="match_parent"
8      android:fitsSystemWindows="true"
9      tools:openDrawer="start"
10     android:id="@+id/drawer_layout">
11
12     <include
13         android:id="@+id/activity_home_base"
14         layout="@layout/activity_home_base"
15     />
16
17     <com.google.android.material.navigation.NavigationView
18         android:id="@+id/nav_view"
19         android:layout_width="wrap_content"
20         android:layout_height="match_parent"
21         android:layout_gravity="start"
22         android:fitsSystemWindows="true"
23         app:headerLayout="@layout/nav_header"
24         app:menu="@menu/nav_menu" />
25 </androidx.drawerlayout.widget.DrawerLayout>
```

Slika 18. DrawerLayout

U *activity_home_base* layout-u kreiran je Toolbar ranije opisan i *activity_home_content* layout (slika 19).

```

1      <?xml version="1.0" encoding="utf-8"?>
2      <androidx.constraintlayout.widget.ConstraintLayout
3          xmlns:android="http://schemas.android.com/apk/res/android"
4          xmlns:app="http://schemas.android.com/apk/res-auto"
5          xmlns:tools="http://schemas.android.com/tools"
6          android:layout_width="match_parent"
7          android:layout_height="match_parent"
8          tools:context=".activities.HomeActivity">
9
10         <!-- AppBarLayout and Toolbar -->
11     > <com.google.android.material.appbar.AppBarLayout...>
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35         <include layout="@layout/activity_home_content" />
36
37     > <com.google.android.material.floatingactionbutton.FloatingActionButton...>
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52     </androidx.constraintlayout.widget.ConstraintLayout>

```

Slika 19. Home Base layout

Unutar `activity_home_content.xml` kreirana je komponenta `androidx.navigation.fragment.NavHostFragment` koja se koristi unutar Android aplikacija kao deo *Android Jetpack Navigation* komponente. Ova komponenta predstavlja kontejner za prikazivanje i upravljanje fragmentima (delovima korisničkog interfejsa) unutar koje se odvija navigacija između različitih ekrana (destinacija) unutar aplikacije.

Glavna svrha *NavHostFragment*-a je pružanje podrške za implementaciju navigacije kroz različite delove aplikacije, omogućavajući prelazak između različitih fragmenata ili destinacija koristeći Android Navigation komponentu.

`app:defaultNavHost="true"`: Ova linija označava ovaj fragment kao "defaultni" NavHost. To znači da će ovaj fragment hvatati sve navigacijske događaje i destinacije.

`app:navGraph="@navigation/base_navigation"`: Ovde se navodi koja XML datoteka sadrži grafički prikaz navigacije (NavGraph) unutar aplikacije koja se koristi za ovaj NavHostFragment (slika 20). `@navigation/base_navigation` referencira se na definiciju grafičke mape navigacije unutar resursa aplikacije (slika 21). Ova mapa definiše putanje i destinacije kroz koje se može navigirati unutar aplikacije.

```

16     <fragment
17         android:id="@+id/fragment_nav_content_main"
18         android:name="androidx.navigation.fragment.NavHostFragment"
19         android:layout_width="match_parent"
20         android:layout_height="match_parent"
21         app:layout_constrainedHeight="true"
22         app:layout_constraintBottom_toBottomOf="parent"
23         app:layout_constraintHorizontal_bias="0.0"
24         app:layout_constraintLeft_toLeftOf="parent"
25         app:layout_constraintRight_toRightOf="parent"
26         app:layout_constraintTop_toTopOf="parent"
27         app:layout_constraintVertical_bias="0.0"
28         app:defaultNavHost="true"
29         app:navGraph="@navigation/base_navigation" />

```

Slika 20. *NavHostFragment*

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <navigation
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      xmlns:tools="http://schemas.android.com/tools"
6      android:id="@+id/base_navigation"
7      app:startDestination="@id/nav_products"
8  >
9      <fragment
10         android:id="@+id/nav_products"
11         android:name="com.example.shopapp.fragments.products.ProductsPageFragment"
12         android:label="Products"
13         tools:layout="@layout/fragment_products_page"
14     />
15
16     <fragment
17         android:id="@+id/nav_new"
18         android:name="com.example.shopapp.fragments.new_product.NewProductFragment"
19         android:label="New product"
20         tools:layout="@layout/fragment_new_product"
21         app:popUpTo="@+id/nav_products"
22         app:popUpToInclusive="true"
23     />

```

Slika 21. *Navigation*

ActionBarDrawerToggle se koristi za povezivanje i upravljanje navigation drawer-om unutar Android aplikacije. *ActionBarDrawerToggle* je klasa koja olakšava sinhronizaciju između navigation drawer-a i

ActionBar-a (ili Toolbar-a) te omogućava otvaranje i zatvaranje navigation drawer-a putem ikone u ActionBar-u ili Toolbar-u.

NavigationController se koristi za upravljanje promenama destinacija unutar Android aplikacije korištenjem Android Navigation komponente. Pomoću *NavController* i *OnDestinationChangedListener*, prati se promena trenutne destinacije (screen-a/fragmenta) unutar aplikacije. Putem *navigate()* metode iz *NavController*-a moguće je navigirati se na specifičan fragment / screen.

Korišćenje Navigation Component-a nudi mnoge prednosti, uključujući automatsko upravljanje back stack-om, podršku za animacije i tranzicije, olakšano upravljanje argumentima i podacima između destinacija, kao i integraciju sa ostalim *Jetpack* komponentama poput *ViewModel*-a i *SafeArgs* za siguran prenos podataka.

AppBarConfiguration odnosi se na konfiguraciju ActionBar-a (ili Toolbar-a) u Android aplikaciji kako bi se omogućila navigacija koristeći Android Navigation komponentu. Takođe, postavlja se bočni meni (navigation drawer) u skladu sa konfiguracijom akcione trake i navigacije. Svaki ID menija prosleđuje se kao skup ID-ova jer svaki meni treba smatrati odredištima najvišeg nivoa.

5. Toasts / Snackbars

Toast je *pop-up* poruka koja automatski nestaje posle određenog vremena. Ovakve poruke korisniku daju povratne informacije da je neka akcija izvršena ili eventualno da je došlo do neke greške.

Na slici se nalazi metoda koja dodeljuje akcije dugmičima iz menija (slika 22 i 23). Kada korisnik klikne na dugme *nav_new* u tom trenutku će se prikazati *toast* sa porukom „New product“, a kada klikne na dugme *nav_logout* prikazaće se poruka „Logout“.

```
85         private void handleNavigation(int id) {  
86             if (id == R.id.nav_products) {  
87                 showToast( text: "Products");  
88             }  
89             } else if (id == R.id.nav_new) {  
90                 showToast( text: "New product");  
91             }  
92             } else if (id == R.id.nav_profile) {  
93                 showToast( text: "Profile");  
94             }  
95             } else if (id == R.id.nav_logout) {  
96                 showToast( text: "Logout");
```

Slika 22. Pozivanje funkcije za prikaz *Toast* poruke

```
126         private void showToast(String text) {  
127             Toast.makeText( context: this, text, Toast.LENGTH_SHORT).show();
```

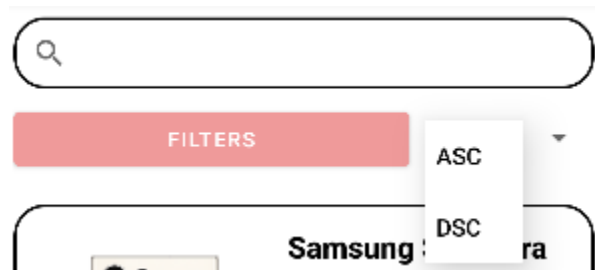
Slika 23. Kreiranje *toast*-a

6. Spinner

Spinner omogućava odabir jedne od više ponuđenih vrednosti. Na slikama 24 i 25 prikazan je izgled ovog elementa.



Slika 24. Izgled *spinner*-a sa izabranom opcijom



Slika 25. Izgled *spinner*-a sa ponuđenim opcijama

Za kreiranje elementa neophodno je definisati ga unutar XML datoteke kao na slici 26.

```
44         <Spinner
45             android:id="@+id/btnSort"
46             android:layout_width="0dp"
47             android:layout_height="wrap_content"
48             android:layout_weight="0.3"
49             android:layout_marginStart="10dp"
50             android:contentDescription="Sort" />
```

Slika 26. Definisanje *spinner*-a

Ponuđene opcije mogu se kreirati unutar *values* foldera kao *sort-array* (slika 27).

```
1         <?xml version="1.0" encoding="utf-8"?>
2         <resources>
3             <string-array name="sort_array">
4                 <item>ASC</item>
5                 <item>DSC</item>
6             </string-array>
7         </resources>
```

Slika 27. Datoteka sa predefinisanim vrednostima *spinner*-a

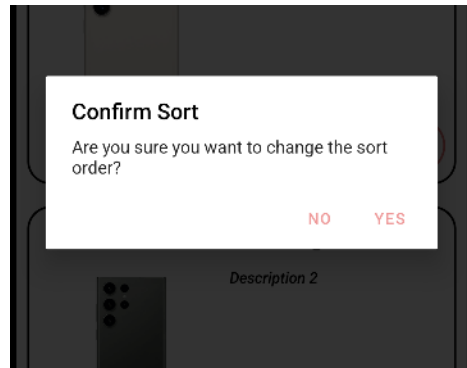
Za povezivanje prethodno prikazanih vrednosti, koriste se adapteri. Na slici 28. iskorišćen je *ArrayAdapter* kom se prosleđuje ugrađen *layout* - *simple_spinner_item* i element sa definisanim vrednostima - *sort_array*. Otvoriti klasu *ProductsPageFragment.java*.

```
97     private void setupSortSpinner() {
98         Spinner spinner = binding.btnSort;
99         ArrayAdapter<String> adapterSpinner = new ArrayAdapter<>(requireActivity(),
100             android.R.layout.simple_spinner_item,
101             getResources().getStringArray(R.array.sort_array));
102         adapterSpinner.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
103         spinner.setAdapter(adapterSpinner);
104
105         final boolean[] userTouched = {false};
106         spinner.setOnTouchListener((View v, MotionEvent event) -> {
107             userTouched[0] = true;
108             return false;
109         });
110
111         spinner.setOnItemClickListener(new AdapterView.OnItemClickListener() {
```

Slika 28. Povezivanje *spinner*-a sa predefinisanim vrednostima

7. Dijalozi

Dijalozi su komponente koje predstavljaju male prozore. Prikazuju se korisniku uglavnom kada se traži potvrda neke akcije ili unos nekih dodatnih podataka. Primer jednostavnog dijaloga dat je na slici 30.



Slika 30. Dijalog sa “da” i “ne” opcijom

Postoje izvedene klase poput *AlertDialog* i *DatePickerDialog* koje nasleđuju baznu klasu *Dialog*. Svaka od ovih klasa poseduje predefinisane *layout*-e i elemente koje je moguće prilagođavati. Na slici 31. prikazana je implementacija *AlertDialog*-a. Pomoću *setMessage* prosleđuje se tekst dijaloga, a *setPositiveButton* i *setNegativeButton* postavljaju dugmad za potvrđivanje i odustajanje. Da bi se dijalog prikazao potrebno je pozvati metodu *show()*.

```
112         spinner.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
113             3 usages
114             int lastPosition = -1;
115             1 usage
116             @Override
117             public void onItemSelected(AdapterView<?> parent, View view, int position, long id) {
118                 if (!userTouched[0] || position == lastPosition) return;
119                 lastPosition = position;
120                 userTouched[0] = false;
121                 String selected = parent.getItemAtPosition(position).toString();
122                 new AlertDialog.Builder(requireActivity())
123                     .setTitle("Confirm Sort")
124                     .setMessage("Are you sure you want to change the sort order?")
125                     .setPositiveButton(text: "Yes", (DialogInterface dialog, int which) -> {
126                         if (selected.equalsIgnoreCase(anotherString: "ASC")) {
127                             productsViewModel.sortByTitleAscending();
128                         } else if (selected.equalsIgnoreCase(anotherString: "DSC")) {
129                             productsViewModel.sortByTitleDescending();
130                         }
131                     })
132                     .setNegativeButton(text: "No", (DialogInterface dialog, int which) -> {
133                         spinner.setSelection(lastPosition);
134                     })
135                     .setCancelable(false)
136                     .show();
137             }
138         });
```

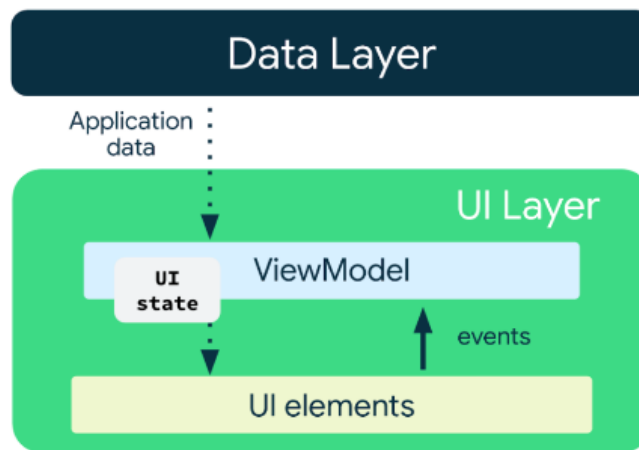
Slika 31. Implementacija *AlertDialog*-a

8. View Model

Upravljanje stanjem u Androidu odnosi se na proces upravljanja i čuvanja stanja aplikacije tokom različitih promena konfiguracije (kao što su rotacija ekrana) i dok je proces živ. Cilj je pružiti besprekorno korisničko iskustvo čuvanjem stanja aplikacije uprkos promenama.

Klasa *ViewModel* (slika 32) je nosilac poslovne logike ili držač stanja na nivou ekrana. *ViewModel* klasa izlaže stanje korisničkom interfejsu i enkapsulira povezanu poslovnu logiku.

Glavna prednost je što kešira stanje i održava ga kroz promene konfiguracije. To znači da vaš korisnički interfejs ne mora ponovo da preuzima podatke prilikom navigacije između fragmenata u istoj aktivnosti, ili prilikom promena konfiguracije, kao što je rotacija ekrana.



Slika 32. Prikaz UI Layer-a

Ključne prednosti klase *ViewModel* su suštinski dve:

1. Omogućava vam da održavate stanje korisničkog interfejsa.
2. Pruža pristup poslovnoj logici.

8.1 Perzistencija (Persistence)

ViewModel je persistence layer i omogućava perzistenciju kako kroz stanje koje *ViewModel* drži, tako i kroz operacije koje *ViewModel* pokreće. Ovo keširanje znači da ne morate ponovo preuzimati podatke kroz uobičajene promene konfiguracije, kao što je rotacija ekrana.

8.2 SavedStateHandle

SavedStateHandle vam omogućava da sačuvate podatke ne samo kroz promene konfiguracije, već i kroz ponovno stvaranje procesa. To jest, pomaže pri obnavljanju UI stanja nakon process

death-a kada sistem rekreira ekran u okviru istog task-a / back stack-a. Ako korisnik force stop-uje aplikaciju ili je “očisti” iz recents na način koji ukloni task (zavisi od scenarija), često nema obnove tog stanja. Takođe, state je ograničen (Bundle) i nije za velike objekte.

Više na linku:

<https://developer.android.com/topic/libraries/architecture/viewmodel/viewmodel-savedstate#java>

8.3 Opseg (Scope)

Opseg (scope) ViewModel-a znači: koliko dugo ViewModel živi i ko ga deli.

Kada instancirate *ViewModel*, prosleđujete mu objekat koji implementira interfejs *ViewModelStoreOwner*. To može biti destinacija u navigacionoj komponenti, grafikon navigacije, aktivnost, fragment ili bilo koji drugi tip koji implementira interfejs. Vaš *ViewModel* je onda ograničen na životni ciklus *ViewModelStoreOwner*-a. Ostaje u memoriji sve dok njegov *ViewModelStoreOwner* trajno ne nestane (dok je aktivnost aktivna, dok je fragment u back stack-u ili dok je navigaciona destinacija i dalje prisutna u back stack-u. Kada *ViewModelStoreOwner* trajno nestane (aktivnost se završi, fragment/destinacija se ukloni iz back stack-a), sistem poziva *onCleared()* i *ViewModel* se uklanja.

Kada se Activity ili Fragment rekreira zbog promene konfiguracije (npr. rotacija ekrana), oni se “unište i ponovo naprave”, ali *ViewModel* se ne gubi jer je vezan za *ViewModelStoreOwner* koji preživljava tu rekreaciju. Zato se **asinhroni posao** (npr. poziv ka mreži, obrada podataka) može nastaviti u istom *ViewModel*-u, a novi UI se samo ponovo poveže na postojeće stanje. Ovo je ključ za očuvanje stanja kroz promene konfiguracije.

8.4 Životni ciklus ViewModel-a

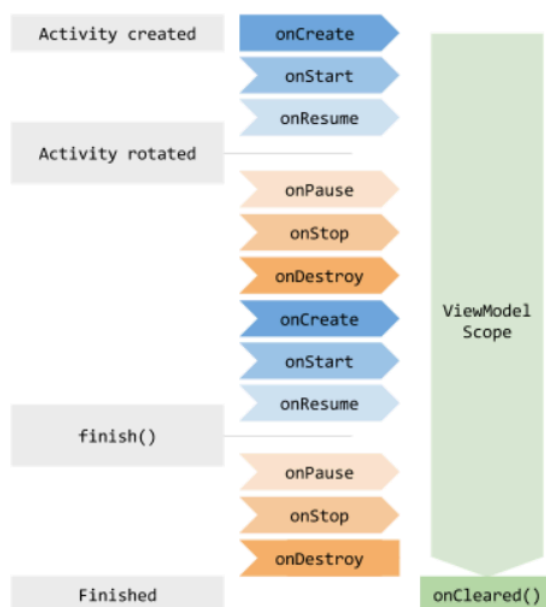
Životni ciklus *ViewModela* je direktno vezan za njegov opseg, odnosno za objekat koji ga poseduje (*ViewModelStoreOwner*).

ViewModel prestaje da postoji (i tada se poziva metoda *onCleared()*) kada njegov *ViewModelStoreOwner* trajno nestane, na primer:

- Activity – kada se aktivnost završi (npr. korisnik izađe nazad ili se pozove *finish()*)
- Fragment – kada se fragment trajno ukloni (npr. uklonjen je iz back stack-a ili je zamenjen tako da se više ne vraća).
- Navigacioni unos (NavBackStackEntry) – kada je destinacija ili *navGraph* uklonjen iz back stack-a.

Ovo čini *ViewModel* odličnim rešenjem za skladištenje podataka koji prežive promene konfiguracije. Tokom promene konfiguracije (npr. rotacija ekrana) sistem uništava i ponovo kreira Activity/Fragment, ali *ViewModel* se ne uništava – on se zadržava i novi UI se samo ponovo poveže na postojeće stanje.

Slika 33 prikazuje kako se Životni ciklus aktivnosti menja tokom rotacije i završavanja, kao i kako *ViewModel* traje paralelno s njom. Ista logika važi i za *ViewModel* vezan za fragment ili navigacioni unos.



Slika 33. Životni ciklus ViewModel-a

Obično *ViewModel* prvi put dobijate u trenutku kada sistem pozove *onCreate()* (npr. preko *ViewModelProvider*). Iako se *onCreate()* može pozvati više puta tokom života aktivnosti (npr. zbog rotacije), *ViewModel* ostaje isti sve dok se aktivnost ne završi trajno i dok se ne pozove *onCleared()*.

8.5 Pristup poslovnoj logici

Iako se većina poslovne logike nalazi u data sloju (npr. u repozitorijumima i use-case-ovima), deo logike prirodno pripada i UI sloju. To je posebno izraženo kada je potrebno:

- kombinovati podatke iz više izvora (više repozitorijuma) kako bi se formiralo UI stanje ekrana,

- primeniti pravila koja su direktno vezana za prikaz i interakciju (npr. validacija unosa, formatiranje, filtriranje i mapiranje podataka za prikaz),
- reagovati na korisničke događaje i odlučiti koje akcije treba pokrenuti.

U tom smislu, *ViewModel* predstavlja centralno mesto za UI-poslovnu logiku: on priprema i izlaže stanje korisničkom interfejsu, a istovremeno obrađuje događaje iz UI-ja (klikovi, promene unosa, osvežavanje) i kada je potrebno, delegira operacije ka nižim slojevima (repozitorijumima, servisima) radi izmene ili preuzimanja podataka. Na taj način UI ostaje jednostavan, dok *ViewModel* preuzima odgovornost za logiku koja upravlja ponašanjem ekrana.

8.6 Implementacija

Sledi primer implementacije *ViewModela* za komponentu koja omogućava korisniku pretragu.

U ovom primeru, odgovornost za sticanje i držanje vrednosti teksta koji se pretražuje u *searchView* komponenti leži na *ViewModel-u*, a ne direktno na Aktivnosti ili Fragmentu.

ViewModel se obično koristi u kombinaciji sa *LiveData* ili nekom drugom *observables* klasom kako bi se osiguralo da UI reaguje na promene podataka. *LiveData* omogućava *ViewModel-u* da automatski ažurira UI komponente kada se podaci promene, čime se osigurava da je korisnički interfejs uvek sinhronizovan sa trenutnim stanjem podataka.

Na slici 34 prikazan je *ProductsPageViewModel*. Promenljiva *searchText* prima String podatak koji predstavlja unetu vrednost unutar *search* polja. *MutableLiveData* je lifecycle-aware observable klasa, što znači da UI komponente (kao što su Activity ili Fragment) mogu posmatrati (observe) promene u ovim podacima i biti automatski obaveštene kada do promene dođe. Posmatranje *LiveData* objekata omogućava UI komponentama da reaguju na promene podataka, ažurirajući se prema potrebi. Prosleđeni primer sadrži dve liste produkata: celu listu i listu filtriranih produkata na osnovu prosleđene vrednosti u polju *searchText*.


```

12 public class ProductsPageViewModel extends ViewModel {
13
14     2 usages
15     private final MutableLiveData<List<Product>> allProducts = new MutableLiveData<>(new ArrayList<>());
16     6 usages
17     private final MutableLiveData<List<Product>> filteredProducts = new MutableLiveData<>(new ArrayList<>());
18     2 usages
19     private String searchText = "";
20
21     1 usage
22     public LiveData<List<Product>> getFilteredProducts() {
23         return filteredProducts;
24     }
25
26     public void setProducts(List<Product> products) {
27         allProducts.setValue(new ArrayList<>(products));
28         applyFilters();
29     }
30
31     2 usages
32     public void setSearchText(String query) {
33         searchText = query == null ? "" : query;
34         applyFilters();
35     }
36 }

```

Slika 34. ProductsPageViewModel

Metoda *applyFilters()* poredi nazive produkata unutar liste sa prosleđenim nazivom u *search* polju i vraća produkte koji se podudaraju.

```

50 private void applyFilters() {
51     List<Product> source = allProducts.getValue();
52     if (source == null) return;
53
54     List<Product> filtered = new ArrayList<>();
55     for (Product p : source) {
56         if (p.getTitle().toLowerCase().contains(searchText.toLowerCase())) {
57             filtered.add(p);
58         }
59     }
60     filteredProducts.setValue(filtered);
61 }
62 }

```

ViewModel obično ne bi trebalo da upućuje na pogled, životni ciklus ili bilo koju klasu koja može da sadrži referencu na kontekst aktivnosti. Pošto je Životni ciklus *ViewModel*-a veći od

korisničkog interfejsa, držanje API-ja koji se odnosi na Životni ciklus u ViewModel-u može izazvati curenje memorije.

Unutar `ProductsPageFragment.java` klase, prikazano je kreiranje instance ViewModel-a (slika 35).

```
27 </> public class ProductsPageFragment extends Fragment {
28
29     8 usages
    private FragmentProductsPageBinding binding;
30     7 usages
    private ProductsPageViewModel productsViewModel;
31     4 usages
    private ProductsListAdapter adapter;
32
33     @Override
34     public View onCreateView(@NonNull LayoutInflater inflater, ViewGroup container,
35                             Bundle savedInstanceState) {
36
37         binding = FragmentProductsPageBinding.inflate(inflater, container, attachToParent: false);
38         productsViewModel = new ViewModelProvider(owner: this).get(ProductsPageViewModel.class);
```

Slika 35. Kreiranje instance ViewModel-a

Zatim se pristupa *ViewModel-u* iz fragmenta na sledeći način:

```
46         productsViewModel.getFilteredProducts().observe(getViewLifecycleOwner(), List<Product> products -> {
47             if (adapter != null) {
48                 adapter.setProducts(products);
49             }
50         });
51
52         return binding.getRoot();
```

Slika 36. Pristupanje viewModelu iz aktivnosti/fragmenta

Posmatranje *LiveData* objekta i ažuriranje UI-a: Ovaj deo koda dodaje observer na *LiveData<List<Product>>* objekat unutar *ProductsPageViewModel*. Svaki put kada dođe do promene podatka unutar *LiveData* objekta (u ovom slučaju *filteredProducts*), ta promena se automatski prosleđuje. Nakon promene ažurira se adapter koji vrši prikaz nove filtrirane liste produkata. Funkcija *getViewLifecycleOwner()* garantuje da se observer povezuje sa Životnim ciklusom vlasnika prikaza fragmenta, što znači da će se observer automatski ukloniti kada fragment više nije vidljiv ili je uništen, sprečavajući time moguće curenje memorije.

9. Domaći

Domaći se nalazi na *Canvas-u* (*canvas.ftn.uns.ac.rs*) na putanji *Вежбе/04 Задачамак.pdf*.

Primer možete preuzeti na sledećem linku:

<https://gitlab.com/mobilne-aplikacije/mobilne-aplikacije>

Za dodatna pitanja možete se obratiti asistentima:

- Milan Jovkić (milan.jovkic@uns.ac.rs)
- Jelena Matković (matkovic.jelena@uns.ac.rs)